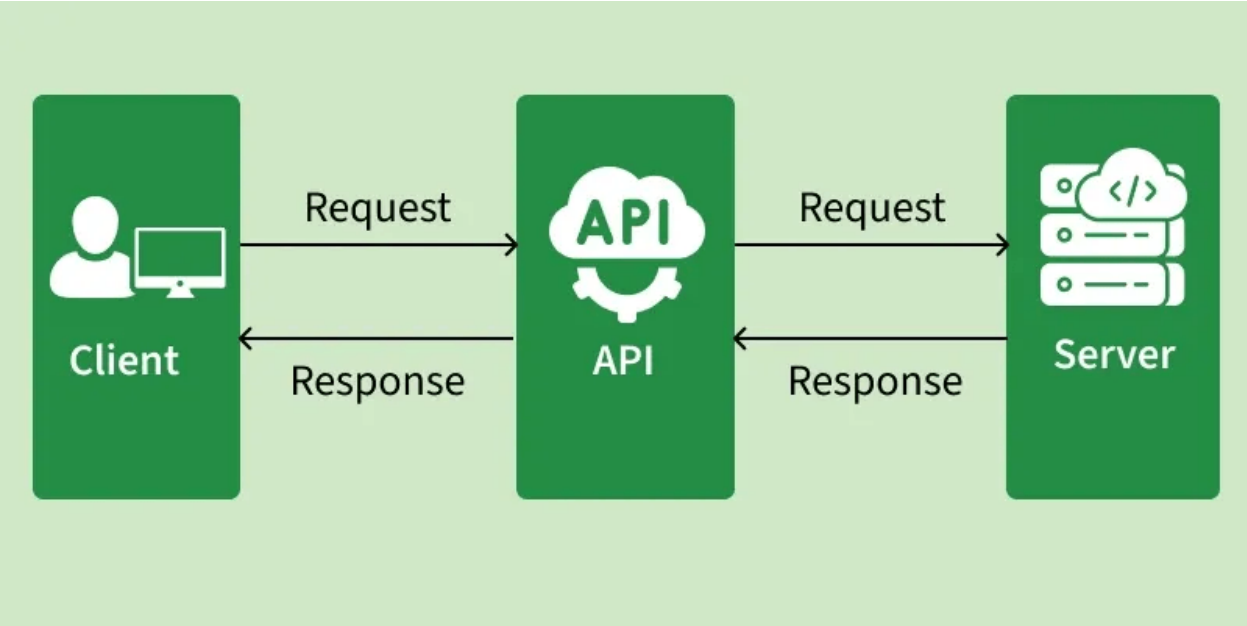
Sole Temperature API

Wilmer Soriano

What is an API? 1
Why use API? 2
Why do so many types of API exist? 3
Sole Temperature API mechanics 4
Security 6

What is an API?

An API (Application Programming Interface) is a way for different software applications to communicate with each other.



Analogy:

1. You (client) request for order (data)
2. The waiter (API) takes the request to kitchen (Server)
3. The kitchen prepares the order
4. The waiter returns the response (the data requested)

Why use API?

APIs are important for building scalable, flexible, and connected programs. That is simple to deploy and maintain.

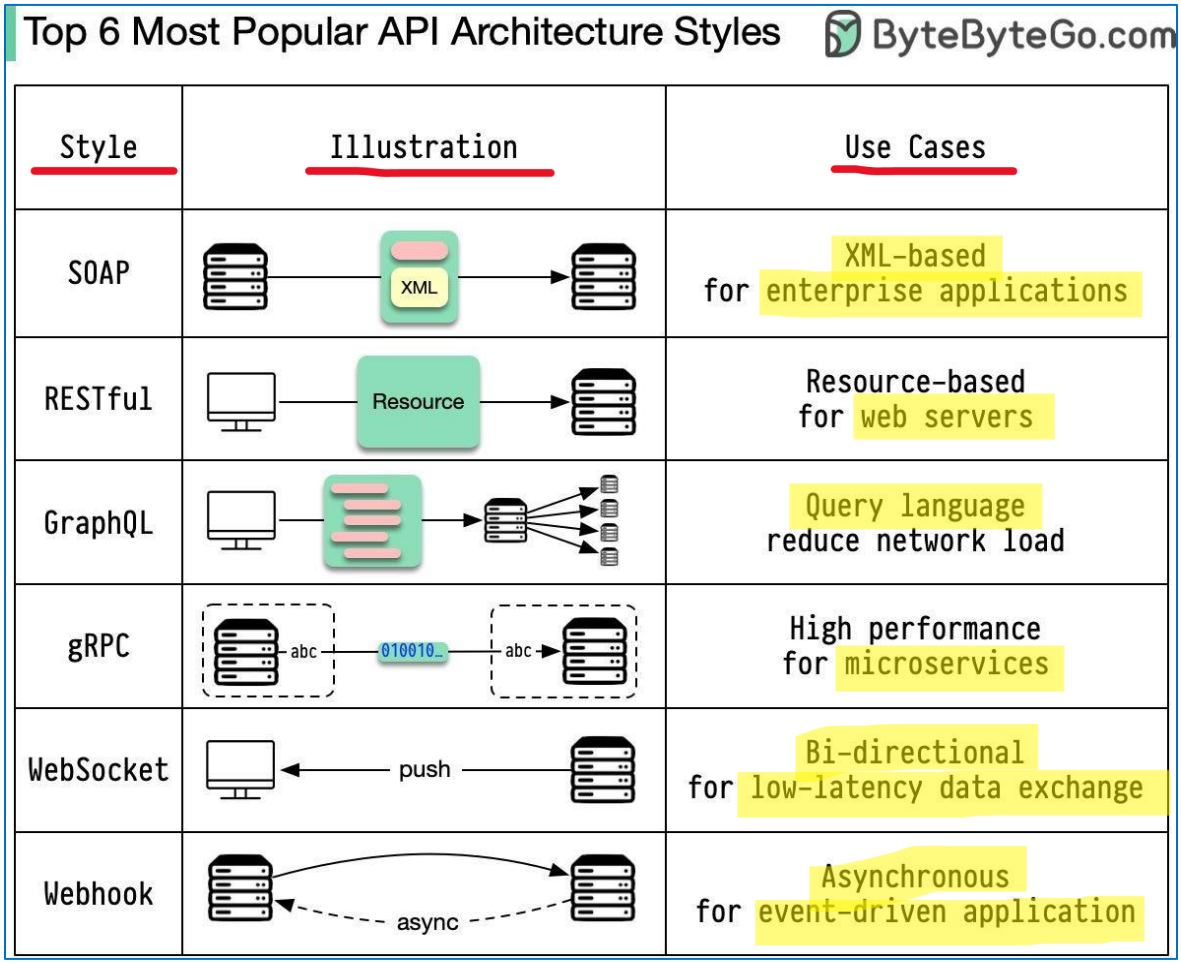
1. **Communication between system**: An API lets different software systems communicate with each other.
   Ex:
   Web/Mobile (Frontend) ---> Java (Backend) ---> MySQL (Database)

2. **Reusability**: APIs are reusable; they can serve multiple clients.
   Ex:
   - Mobile apps
   - Web app
   - Desktop app

3. **Secure**: API provides security in terms of indirect access.
   Ex:
   Client -> API -> Database
   The client does not have direct access to the database, preventing miss use.

Why do so many types of API exist?

Multiple API architecture exists to define how systems communicate and exchange data based on application needs.



REST - Most popular - uses HTTP request and returns a JSON format data. Common in Web-dev

SOAP - Old enterprise, but still common – use only XML, but has strong security. Common in banks/enterprise systems.

Sole Temperature API mechanics

The Sole Temperature uses a REST API. Specifically, it is calling the OpenWeather API, which is a REST API.

In this case, my Python program acts as a client, and the weather service is the API server.
Flow:

Python Script (client)

↓

HTTP Request

↓

OpenWeather REST API

↓

JSON Response

1. To request data from a web server, I use python library: **requests**
   - **Requests** allow you to send HTTP/1.1 requests extremely easily. There's no need to manually add query strings to your URLs, or to form-encode your POST data.
     Ex:
     r = requests.**get**('https://api.github.com/user')

2. Using the method: r**.status_code** . You can check HTTP status code such as: 200(good), 404 (not found), 400 (bad request), etc...
3. Extract the data into JSON format:
   data = r.**json()**

```python

CityName = "Dallas"

API_KEY = os.getenv("OPEN_API_KEY")

URL = "https://api.openweathermap.org/data/2.5/weather"

CALL = f"{URL}?q={CityName}&appid={API_KEY}&units=imperial"

response = requests.get(CALL)

if response.status_code == 200:
    data = response.json()
```

NOTE: You'll need an API key. An API Key is a unique secret code used to identify and authenticate an application when it accesses an API. It tells the server who is making the request, For: Security, Rate limits, Billing, etc... (Depends on the server service)

Security

In terms of security for Sole Temperature, I implemented 2 main sources.

1. Environment Variable
   - What is an Environment Variable?
     An environment variable is a versatile set value that is used throughout the program and determines the request of a program on each device. Simply speaking, I pair my password with a variable name. This variable name can be used through the program without revealing the password.
   - For my project I used the env. File to store the following:
     `OPEN_API_KEY = “my person API key value here”`
   - Now my program can look for a .env file and inside the file look for variable name “OPEN_API_KEY

```
def find_env():
    base_dir = Path(__file__).resolve().parent.parent
    """
        __ __ [redacted]
        [redacted]
        [redacted]
        [redacted]
    """
    # creates the path to .env file
    load_path = base_dir / ".env"

    return load_path

def fetch_data():
    # Loads the variables from .env into environment
    load_dotenv(find_env())

    # Location: Dallas, Texas, US
    CityName = "Dallas"

    # using .env, call the variable name, keeping API Key a Secret
    API_KEY = os.getenv("OPEN_API_KEY")
```

   - Now I can prevent many users from using other credentials and prevent overuse of a singler API key subscription.

2. HTTP error handling
   - After requesting a HTTP response from the URL created. I added multiple exception handlings to extract the correct data and prevent unknown data extraction.

   a. First, I made sure to request the data it should take 3 seconds max to connect to server and 5 seconds to wait for a response from server. Otherwise, throw an error. After multiple test trials, this is the max time I give my python to get a response back.
   b. There are 4 exceptions as follows: HTTP error that handles 400s error scenarios, Time Out Scenario, connection error, and finally the general error catch which can vary.

c. Finally, wrap the try/exception handling. I still checked if the HTTP status code is 200, otherwise it's a server/client issue. Even then I made use after converting the raw data into json file, the temperatures values is format in a specified manner. Such as data[main][temp], [temp_max], and [temp_min]. If these values do not exist, I would get a python IndexError. Indicating this is not the correct data.

a,b are 1st red box \/

```python
    try:
        # should return HTTP response if everything is correct
        response = requests.get(CALL, timeout=(3, 5)) # 3 sec MAX time to connect, 5 sec MAX time to wait for response
    except requests.exceptions.HTTPError as errh:
        print("HTTP Error:", errh.args[0])
    except requests.exceptions.ReadTimeout as errrt:
        print("Time out")
    except requests.exceptions.ConnectionError as conerr:
        print("Connection error")
    except requests.exceptions.RequestException as errex:
        print("General Exception: Unknown error")

    if response.status_code == 200:
        data = response.json()

        temp = float(data["main"]["temp"])
        h_temp = int(data["main"]["temp_max"])
        l_temp =  int(data["main"]["temp_min"])

        return temp, h_temp, l_temp
    else:
        print("Error:", response.status_code)

if __name__ == "__main__":
    temp, h_temp, l_temp = fetch_data()
    print(temp, h_temp, l_temp)
```

c is 2nd red box /\